# تحليل وتصميم الخوارزميات

## المحاضرة السابعة: خوارزميات الترتيب (الجزء الثالث) - الترتيب السريع والترتيب بالأساس

### مقدمة

في محاضرتنا السابقة. حتى الآن، درسنا خوارزميات ترتيب بسيطة مثل $O(n^2)$ وخوارزميات محسنة مثل ترتيب شيل. اليوم، سنتعمق في خوارزمية "فرق تسد " (Divide and Conquer)بالغة الأهمية وهي **الترتيب السريع(Quick Sort)** ، والتي تعتبر من أسرع خوارزميات الترتيب المعتمدة على المقارنة في الحالة المتوسطة. ثم سنستكشف طريقة تفكير مختلفة تماماً مع **الترتيب بالأساس (Radix Sort)**.

### 1.الترتيب السريع(Quick Sort)

**الفكرة الأساسية :**الترتيب السريع هو مثال كلاسيكي على استراتيجية **"فرق تسد " (Divide and Conquer)**، والتي تتبع ثلاث خطوات:

1. **فرّق :(Divide)** اختر عنصراً من القائمة يُسمى **"المحور .(Pivot) "**ثم أعد ترتيب القائمة بحيث تأتي كل العناصر الأصغر من المحور قبله، وكل العناصر الأكبر من المحور بعده. بعد هذه الخطوة، يكون المحور قد وصل إلى مكانه النهائي الصحيح. هذه العملية تسمى **التقسيم.(Partitioning)**
2. **تَسُد :(Conquer)** قم بتطبيق خوارزمية الترتيب السريع بشكل **استدعاء ذاتي (Recursively)** على القائمتين الفرعيتين اللتين تكونتا على جانبي المحور (قائمة العناصر الأصغر، وقائمة العناصر الأكبر).
3. **اجمع :(Combine)** لا توجد خطوة تجميع هنا! بمجرد أن يتم ترتيب القائمتين الفرعيتين، تكون القائمة بأكملها مرتبة تلقائياً.

**خطوات الخوارزمية:**

1. **اختر المحور :(Pivot)** يمكن اختيار أي عنصر كمحور. الطرق الشائعة هي اختيار العنصر الأول، أو الأخير، أو عنصر عشوائي، أو العنصر الأوسط.
2. **قم بعملية التقسيم :(Partition)** أعد ترتيب القائمة حول المحور. هناك طرق عديدة لتنفيذ ذلك، ولكن الهدف النهائي واحد] :عناصر أصغر من المحور] [المحور] [عناصر أكبر من المحور].

3. **كرر العملية :(Recurse)** قم باستدعاء الترتيب السريع على القائمة الفرعية اليسرى (الأصغر) والقائمة الفرعية اليمنى (الأكبر).
4. **الشرط الأساسي للتوقف :**تتوقف عملية الاستدعاء الذاتي عندما تصل إلى قائمة فرعية تحتوي على عنصر واحد أو لا تحتوي على أي عناصر، لأن مثل هذه القائمة تعتبر مرتبة بطبيعتها.

**مثال عملي :**لنرتب القائمة [7, 2, 1, 6, 8, 5, 3, 4]باختيار العنصر الأخير كمحور.

- **الدورة الأولى:**
  - القائمة .[7, 2, 1, 6, 8, 5, 3, 4] :**المحور هو .4**
  - **التقسيم :**بعد إعادة الترتيب، كل ما هو أصغر من 4 يأتي قبله، وكل ما هو أكبر يأتي بعده.
  - النتيجة) .[3, 2, 1] **[4]** [8, 5, 7, 6] :الآن 4في مكانه النهائي.(
- **الدورة الثانية (استدعاء ذاتي):**
  - نرتب القائمة اليسرى.[3, 2, 1] :
  - نرتب القائمة اليمنى.[8, 5, 7, 6] :
- تستمر هذه العملية على القوائم الفرعية الأصغر حتى تصبح القائمة بأكملها مرتبة.[1, 2, 3, 4, 5, 6, 7, 8] :

**تحليل الأداء:(Quick Sort)**

- **تعقيد الوقت:**
  - **الحالة الأفضل والمتوسطة :(Best/Average Case)** عندما ينجح المحور في كل مرة في تقسيم القائمة إلى نصفين متساويين تقريباً. في هذه الحالة، يكون التعقيد ممتازاً وهو .**O(n log n)**
  - **الحالة الأسوأ :(Worst Case)** عندما يتم اختيار المحور بشكل سيء في كل مرة (مثلاً، اختيار أصغر أو أكبر عنصر في قائمة مرتبة)، مما يؤدي إلى تقسيم غير متوازن n-1) عنصر في جهة وصفر في جهة أخرى). هذا يحول الخوارزمية إلى أداء شبيه بالترتيب بالإضافة، ويصبح التعقيد .**$O(n^2)$**

- **تعقيد المساحة** O(log n) :بسبب عمق الاستدعاءات الذاتية (Recursion call stack).

**.2الترتيب بالأساس(Radix Sort)**

**الفكرة الأساسية :**هذه الخوارزمية فريدة من نوعها لأنها **لا تعتمد على مقارنة العناصر ببعضها البعض** .بدلاً من ذلك، تقوم بترتيب الأرقام عن طريق تجميعها بناءً على **خاناتها(digits)** ، بدءاً من الخانة الأقل أهمية (الآحاد) وصولاً إلى الخانة الأكثر أهمية.

في سياق الخوارزميات، يشير مصطلح **"الدلو (Bucket) "**بشكل خاص إلى حاوية أو مجموعة مؤقتة تُستخدم لتخزين وتصنيف العناصر أثناء عملية **الترتيب (Sorting)**.هذا المصطلح مرتبط ارتباطًا وثيقًا بخوارزميات الترتيب التي لا تعتمد على المقارنة.(Non-Comparison Sorts)

**شروط الاستخدام:**

- تعمل هذه الخوارزمية بشكل أساسي على الأعداد الصحيحة.
- تتطلب أن تكون البيانات ذات طول ثابت أو يمكن جعلها كذلك.

**خطوات الخوارزمية (لترتيب أعداد صحيحة):**

1. أوجد العدد الذي يحتوي على أكبر عدد من الخانات لتحديد عدد الدورات المطلوبة.
2. أنشئ 10 "دلاء(buckets) "، واحد لكل خانة من 0 إلى 9.
3. **الدورة الأولى (خانة الآحاد):**

1. مر على كل رقم في القائمة.
2. ضع كل رقم في الدلو الذي يتوافق مع خانة الآحاد الخاصة به.

4. أعد تجميع الأرقام في قائمة واحدة جديدة عن طريق جمع محتويات الدلاء بالترتيب (من دلو 0 إلى دلو 9). القائمة الآن مرتبة بناءً على خانة الآحاد.
5. **الدورة الثانية (خانة العشرات):**

0. كرر العملية: مر على القائمة الجديدة، وضع كل رقم في الدلو الذي يتوافق مع خانة العشرات الخاصة به.

6. أعد تجميع الأرقام من الدلاء مرة أخرى.

7. استمر في هذه العملية لكل الخانات (مئات، آلاف، إلخ) حتى تصل إلى الخانة الأكثر أهمية. بعد الدورة الأخيرة، ستكون القائمة مرتبة بالكامل.

**مثال عملي :**لنرتب القائمة [170, 45, 75, 90, 802, 24, 2, 66]

- **الدورة الأولى (بناءً على الآحاد):**
  - <- **0**9 ,**0**17دلو 0
  - <- **2** ,**2**80دلو 2
  - <- **4**2دلو 4
  - <- **5**7 ,**5**4دلو 5
  - <- **6**6دلو 6
  - القائمة بعد التجميع[170, 90, 802, 2, 24, 45, 75, 66] :
- **الدورة الثانية (بناءً على العشرات):**
  - <- 2**0** ,2**0**8دلو 0
  - <- 4**2**دلو 2
  - <- 5**4**دلو 4
  - <- 6**6**دلو 6
  - <- 5**7** ,0**7**1دلو 7
  - <- 0**9**دلو 9
  - القائمة بعد التجميع[802, 2, 24, 45, 66, 170, 75, 90] :
- **الدورة الثالثة (بناءً على المئات):**
  - <- 002, 024, 045, 066, 075, 090دلو 0
  - <- 70**1**دلو 1
  - <- 02**8**دلو 8
  - القائمة النهائية بعد التجميع[2, 24, 45, 66, 75, 90, 170, 802] :

**تحليل الأداء:(Radix Sort)**

- **تعقيد الوقت** ((O(d * (n + b :حيث:
  - dهو عدد الخانات في أكبر رقم.
  - nهو عدد العناصر في القائمة.
  - bهو الأساس المستخدم (في حالتنا 10، لـ 10 دلاء).
  - إذا كان dصغيراً وثابتاً، يمكن تبسيط التعقيد إلى **O(n)**، مما يجعله أسرع نظرياً من أي خوارزمية ترتيب معتمدة على المقارنة.
- **تعقيد المساحة**(O(n + b :، لأننا نحتاج إلى ذاكرة إضافية لتخزين الدلاء.